

FILE WITH CODE FOR HOME WORK 4

```
# Load the dataset

cat("Please select your CSV file from the dialog box...\n")

study_data <- read.csv(file.choose(), stringsAsFactors = FALSE)


# Display dataset information

cat("\n" , paste(rep("=", 40), collapse = ""), "\n")

cat("DATA OVERVIEW\n")

cat(paste(rep("=", 40), collapse = ""), "\n")

cat("Number of rows (samples):", nrow(study_data), "\n")

cat("Number of columns (variables):", ncol(study_data), "\n")

cat("\nVariable names:\n")

print(names(study_data))


# -----

# SECTION 2: VARIABLE CLASSIFICATION

# -----


# Define hormone variables (used for logistic regression)

hormone_list <- c(

  "hormone1", "hormone2", "hormone3", "hormone4",

  "hormone5", "hormone6", "hormone7", "hormone8",
```

```
"hormone10_generated"  
)
```

```
# Define lipid profile variables
```

```
lipid_list <- c("lipids1", "lipids2", "lipids3", "lipids4", "lipids5")
```

```
# Define lipid peroxidation markers
```

```
peroxidation_list <- c("lipid_pero1", "lipid_pero2", "lipid_pero3", "lipid_pero4",  
"lipid_pero5")
```

```
# Define antioxidant markers
```

```
antioxidant_list <- c("antioxidant1", "antioxidant2", "antioxidant3", "antioxidant4",  
"antioxidant5")
```

```
# Define other clinical variables
```

```
clinical_list <- c("carb_metabolism", "factor_eth", "factor_h", "factor_pcos",  
"factor_prl")
```

```
# Keep only variables that actually exist in the dataset
```

```
hormone_list <- hormone_list[hormone_list %in% names(study_data)]
```

```
lipid_list <- lipid_list[lipid_list %in% names(study_data)]
```

```
peroxidation_list <- peroxidation_list[peroxidation_list %in% names(study_data)]
```

```
antioxidant_list <- antioxidant_list[antioxidant_list %in% names(study_data)]
```

```
clinical_list <- clinical_list[clinical_list %in% names(study_data)]
```

```

# Combine all non-hormone variables for correlation analysis

all_other_variables <- unique(c(lipid_list, peroxidation_list, antioxidant_list,
clinical_list))

cat("\n" , paste(rep("=", 40), collapse = ""), "\n")

cat("VARIABLES IDENTIFIED IN DATASET\n")

cat(paste(rep("=", 40), collapse = ""), "\n")

cat("Hormones:", length(hormone_list), "variables\n")

cat("Lipids:", length(lipid_list), "variables\n")

cat("Lipid Peroxidation:", length(peroxidation_list), "variables\n")

cat("Antioxidants:", length(antioxidant_list), "variables\n")

cat("Clinical factors:", length(clinical_list), "variables\n")


# -----

# SECTION 3: CORRELATION ANALYSIS WITH PERMUTATION TESTS

# -----


cat("\n\n" , paste(rep("=", 40), collapse = ""), "\n")

cat("PART 1: CORRELATION ANALYSIS\n")

cat(paste(rep("=", 40), collapse = ""), "\n")


# Function to compute correlation with permutation-based p-value

```

```

permutation_correlation <- function(var_x, var_y, n_permutations = 1000) {
  # Remove missing value pairs
  complete_pairs <- complete.cases(var_x, var_y)
  var_x_clean <- var_x[complete_pairs]
  var_y_clean <- var_y[complete_pairs]

  # Require at least 5 observations
  if (length(var_x_clean) < 5) {
    return(c(NA, NA, length(var_x_clean)))
  }

  # Calculate original Spearman correlation
  original_corr <- cor(var_x_clean, var_y_clean, method = "spearman")

  # Perform permutation test
  set.seed(123) # For reproducibility
  permutation_corrs <- numeric(n_permutations)

  for (iter in 1:n_permutations) {
    shuffled_y <- sample(var_y_clean)
    permutation_corrs[iter] <- cor(var_x_clean, shuffled_y, method = "spearman")
  }

```

```

# Calculate two-tailed p-value

p_value <- mean(abs(permutation_corrs) >= abs(original_corr))

return(c(original_corr, p_value, length(var_x_clean)))
}

cat("\nComputing pairwise correlations...\n")

# Initialize results table
correlation_table <- data.frame(
  Variable_A = character(),
  Variable_B = character(),
  Sample_Size = integer(),
  Correlation_Coefficient = numeric(),
  P_Value = numeric(),
  Significance_Flag = character(),
  stringsAsFactors = FALSE
)

# Calculate all pairwise correlations
for (idx_i in 1:(length(all_other_variables) - 1)) {
  for (idx_j in (idx_i + 1):length(all_other_variables)) {

```

```

cor_result <- permutation_correlation(
  study_data[[all_other_variables[idx_i]]],
  study_data[[all_other_variables[idx_j]]]
)

# Assign significance stars
significance <- ""
if (!is.na(cor_result[2])) {
  if (cor_result[2] < 0.001) significance <- "****"
  else if (cor_result[2] < 0.01) significance <- "***"
  else if (cor_result[2] < 0.05) significance <- "**"
}

correlation_table <- rbind(correlation_table, data.frame(
  Variable_A = all_other_variables[idx_i],
  Variable_B = all_other_variables[idx_j],
  Sample_Size = cor_result[3],
  Correlation_Coefficient = round(cor_result[1], 4),
  P_Value = round(cor_result[2], 6),
  Significance_Flag = significance,
  stringsAsFactors = FALSE
))
}

```

```
}
```

```
# Remove rows with missing correlations
```

```
correlation_table <-  
correlation_table[!is.na(correlation_table$Correlation_Coefficient), ]
```

```
# Sort by absolute correlation strength (strongest first)
```

```
correlation_table <- correlation_table[order(-  
abs(correlation_table$Correlation_Coefficient)), ]
```

```
# Display top correlations
```

```
cat("\nTop 30 strongest correlations:\n")
```

```
print(head(correlation_table, 30))
```

```
# Save complete correlation table
```

```
write.csv(correlation_table, "01_correlation_analysis_results.csv", row.names =  
FALSE)
```

```
cat("\n Correlation table saved to '01_correlation_analysis_results.csv'\n")
```

```
# Summarize correlation strength distribution
```

```
strong_pairs <-  
correlation_table[abs(correlation_table$Correlation_Coefficient) >= 0.5, ]
```

```
moderate_pairs <-  
correlation_table[abs(correlation_table$Correlation_Coefficient) >= 0.3 &
```

```

        abs(correlation_table$Correlation_Coefficient) < 0.5, ]

weak_pairs <- correlation_table[abs(correlation_table$Correlation_Coefficient) <
0.3, ]

significant_pairs <- correlation_table[correlation_table$P_Value < 0.05, ]


cat("\nCorrelation Strength Summary:\n")

cat(" Strong correlations ( $|r| \geq 0.5$ ):", nrow(strong_pairs), "pairs\n")

cat(" Moderate correlations ( $0.3 \leq |r| < 0.5$ ):", nrow(moderate_pairs), "pairs\n")

cat(" Weak correlations ( $|r| < 0.3$ ):", nrow(weak_pairs), "pairs\n")

cat(" Statistically significant ( $p < 0.05$ ):", nrow(significant_pairs), "pairs\n")


# -----

# SECTION 4: REGRESSION ANALYSIS WITH BIC MODEL SELECTION

# -----


cat("\n\n" , paste(rep("=", 40), collapse = ""), "\n")

cat("PART 2: REGRESSION ANALYSIS\n")

cat(paste(rep("=", 40), collapse = ""), "\n")


# Function to select best regression model using BIC criterion

select_best_model_by_bic <- function(dataset, candidate_predictors,
target_response) {

```

```

# Remove missing values

complete_dataset <- dataset[complete.cases(dataset[, c(candidate_predictors,
target_response)]), ]

if (nrow(complete_dataset) < 10) {

  return(list(model = NULL, error_message = "Insufficient data (need ≥10
complete cases)"))

}

# Limit number of predictors to avoid overfitting

max_allowed_predictors <- min(length(candidate_predictors),
floor(nrow(complete_dataset) / 5))

if (length(candidate_predictors) > max_allowed_predictors) {

  # Select predictors with strongest correlation to the response

  correlation_with_response <- sapply(candidate_predictors, function(pred) {

    cor(complete_dataset[[target_response]], complete_dataset[[pred]], use =
"complete.obs")

  })

  candidate_predictors <- names(sort(abs(correlation_with_response), decreasing
= TRUE))[1:max_allowed_predictors]

}

# Attempt to fit full model with stepwise selection

```

```

if (length(candidate_predictors) >= 1) {

  full_formula <- as.formula(paste(target_response, "~",
paste(candidate_predictors, collapse = " + ")))

  full_model <- tryCatch(lm(full_formula, data = complete_dataset), error =
function(e) NULL)

  if (!is.null(full_model)) {

    # Perform stepwise selection using BIC

    best_model <- step(full_model, direction = "both", trace = 0,
                        k = log(nrow(complete_dataset)))

    return(list(model = best_model, error_message = NULL))

  }

}

# If full model fails, try simple single-predictor models

best_bic_value <- Inf

best_single_model <- NULL

for (predictor in candidate_predictors) {

  simple_formula <- as.formula(paste(target_response, "~", predictor))

  simple_model <- tryCatch(lm(simple_formula, data = complete_dataset), error =
function(e) NULL)

  if (!is.null(simple_model)) {

```

```
current_bic <- BIC(simple_model)
if (current_bic < best_bic_value) {
  best_bic_value <- current_bic
  best_single_model <- simple_model
}
}
}
```

```
if (!is.null(best_single_model)) {
  return(list(model = best_single_model, error_message = NULL))
} else {
  return(list(model = NULL, error_message = "Unable to fit any model"))
}
}
```

```
# Initialize regression results summary
regression_summary_table <- data.frame(
  Response_Variable = character(),
  Variable_Category = character(),
  R_Squared = numeric(),
  Adjusted_R_Squared = numeric(),
  BIC_Value = numeric(),
  AIC_Value = numeric(),
```

```
Predictor_Count = integer(),  
Selected_Predictors = character(),  
Strongest_Predictor = character(),  
Strongest_Coefficient = numeric(),  
Strongest_Pvalue = numeric(),  
stringsAsFactors = FALSE  
)
```

```
# Function to analyze a group of response variables
```

```
analyze_response_variables <- function(dataset, response_vars, category_name,  
all_predictors) {
```

```
  for (response_var in response_vars) {  
    cat("\n" , paste(rep("-", 40), collapse = ""), "\n")  
    cat("Analyzing:", response_var, "(", category_name, ") \n")  
    cat(paste(rep("-", 40), collapse = ""), "\n")
```

```
    # Exclude the current response from predictors
```

```
    available_predictors <- all_predictors[all_predictors != response_var]
```

```
    if (length(available_predictors) == 0) {
```

```
      cat(" No predictors available for this variable\n")
```

```
      next
```

```
}
```

```
# Find best model
```

```
modeling_result <- select_best_model_by_bic(dataset, available_predictors,  
response_var)
```

```
if (!is.null(modeling_result$model)) {
```

```
  fitted_model <- modeling_result$model
```

```
  model_summary <- summary(fitted_model)
```

```
# Extract coefficient information
```

```
model_coefficients <- coef(model_summary)
```

```
# Identify the strongest predictor (excluding intercept)
```

```
if (nrow(model_coefficients) > 1) {
```

```
  coefficients_no_intercept <- model_coefficients[-1, , drop = FALSE]
```

```
  if (nrow(coefficients_no_intercept) > 0) {
```

```
    absolute_coefficients <- abs(coefficients_no_intercept[, 1])
```

```
    best_predictor_idx <- which.max(absolute_coefficients)
```

```
    strongest_predictor
```

```
    rownames(coefficients_no_intercept)[best_predictor_idx]
```

```
    strongest_coefficient <- coefficients_no_intercept[best_predictor_idx, 1]
```

```
    strongest_pvalue <- coefficients_no_intercept[best_predictor_idx, 4]
```

```
<-
```

```

    } else {
      strongest_predictor <- "none"
      strongest_coefficient <- NA
      strongest_pvalue <- NA
    }
  } else {
    strongest_predictor <- "none"
    strongest_coefficient <- NA
    strongest_pvalue <- NA
  }

# Store results

regression_summary_table <-<- rbind(regression_summary_table, data.frame(
  Response_Variable = response_var,
  Variable_Category = category_name,
  R_Squared = round(model_summary$r.squared, 4),
  Adjusted_R_Squared = round(model_summary$adj.r.squared, 4),
  BIC_Value = round(BIC(fitted_model), 2),
  AIC_Value = round(AIC(fitted_model), 2),
  Predictor_Count = length(coef(fitted_model)) - 1,
  Selected_Predictors = paste(names(coef(fitted_model))[-1], collapse = ", "),
  Strongest_Predictor = strongest_predictor,
  Strongest_Coefficient = round(strongest_coefficient, 4),

```

```

    Strongest_Pvalue = round(strongest_pvalue, 6),
    stringsAsFactors = FALSE
))

cat("  Model fit - R-squared:", round(model_summary$r.squared, 4), "\n")
cat("  Model quality - BIC:", round(BIC(fitted_model), 2), "\n")
cat("  Predictors selected:", length(coef(fitted_model)) - 1, "\n")

# Save detailed model output
sink(paste0("02_regression_model_", response_var, "_details.txt"))
cat("Regression Analysis Results for:", response_var, "\n")
cat(paste(rep("=", 40), collapse = ""), "\n\n")
print(model_summary)
cat("\n\nModel Selection Criteria:\n")
cat("BIC (Bayesian Information Criterion):", BIC(fitted_model), "\n")
cat("AIC (Akaike Information Criterion):", AIC(fitted_model), "\n")
cat("\nFinal Model Formula:\n")
print(formula(fitted_model))
sink()

} else {
  cat("  Cannot fit model -", modeling_result$error_message, "\n")
}

```

```
}  
}
```

```
# Combine all potential predictors
```

```
all_predictor_variables <- unique(c(lipid_list, peroxidation_list, antioxidant_list,  
clinical_list))
```

```
# Run regression analysis for each variable group
```

```
if (length(lipid_list) > 0) {
```

```
  cat("\n\n=== LIPID VARIABLES ANALYSIS ===\n")
```

```
  analyze_response_variables(study_data, lipid_list, "Lipid",  
all_predictor_variables)
```

```
}
```

```
if (length(peroxidation_list) > 0) {
```

```
  cat("\n\n=== LIPID PEROXIDATION MARKERS ANALYSIS ===\n")
```

```
  analyze_response_variables(study_data, peroxidation_list, "Lipid Peroxidation",  
all_predictor_variables)
```

```
}
```

```
if (length(antioxidant_list) > 0) {
```

```
  cat("\n\n=== ANTIOXIDANT MARKERS ANALYSIS ===\n")
```

```
  analyze_response_variables(study_data, antioxidant_list, "Antioxidant",  
all_predictor_variables)
```

```
}
```

```
# Save regression summary and display best models
```

```
if (nrow(regression_summary_table) > 0) {
```

```
  regression_summary_table                                     <-  
  regression_summary_table[order(regression_summary_table$BIC_Value), ]
```

```
  write.csv(regression_summary_table, "02_regression_all_models_summary.csv",  
    row.names = FALSE)
```

```
  cat("\n\n", paste(rep("=", 40), collapse = ""), "\n")
```

```
  cat("REGRESSION MODEL SUMMARY (Best models by BIC)\n")
```

```
  cat(paste(rep("=", 40), collapse = ""), "\n")
```

```
  print(head(regression_summary_table, 10))
```

```
# Identify the overall best model
```

```
best_model_index <- which.min(regression_summary_table$BIC_Value)
```

```
cat("\n BEST MODEL OVERALL:\n")
```

```
cat("                Response                variable:",  
  regression_summary_table$Response_Variable[best_model_index], "\n")
```

```
cat("    R-squared:", regression_summary_table$R_Squared[best_model_index],  
  "\n")
```

```
cat("  BIC:", regression_summary_table$BIC_Value[best_model_index], "\n")
```

```
cat("                Selected                predictors:",  
  regression_summary_table$Selected_Predictors[best_model_index], "\n")
```

```

}

# -----

# SECTION 5: LOGISTIC REGRESSION FOR BINARY OUTCOME

# -----

cat("\n\n" , paste(rep("=", 40), collapse = ""), "\n")

cat("PART 3: LOGISTIC REGRESSION FOR OUTCOME PREDICTION\n")

cat(paste(rep("=", 40), collapse = ""), "\n")


# Verify required columns exist

if (!"outcome" %in% names(study_data)) {

  cat("ERROR: Dataset missing 'outcome' column!\n")

  cat("Available columns:", paste(names(study_data), collapse = ", "), "\n")

} else if (length(hormone_list) == 0) {

  cat("ERROR: No hormone variables found for logistic regression!\n")

} else {


# Prepare data for logistic regression

logistic_dataset <- study_data[complete.cases(study_data[, c("outcome",
hormone_list)]), ]


cat("\nLogistic regression dataset:\n")

```

```

cat(" Complete cases:", nrow(logistic_dataset), "\n")

cat(" Outcome distribution:\n")

print(table(logistic_dataset$outcome))


if (nrow(logistic_dataset) < 10) {
  cat("ERROR: Insufficient data (need at least 10 complete cases)\n")
} else {

# -----

# MODEL 1: Simple logistic regression (one hormone at a time)

# -----


cat("\n" , paste(rep("-", 40), collapse = ""), "\n")
cat("MODEL 1: Simple Logistic Regression (Single Hormone)\n")
cat(paste(rep("-", 40), collapse = ""), "\n")


simple_logistic_models <- data.frame(
  Hormone = character(),
  Coefficient = numeric(),
  Standard_Error = numeric(),
  Odds_Ratio = numeric(),
  CI_Lower_95 = numeric(),
  CI_Upper_95 = numeric(),

```

```
P_Value = numeric(),  
AIC_Value = numeric(),  
BIC_Value = numeric(),  
stringsAsFactors = FALSE  
)
```

```
for (hormone in hormone_list) {  
  logistic_model <- glm(outcome ~ get(hormone), data = logistic_dataset, family  
= binomial)
```

```
  coefficient_summary <- summary(logistic_model)$coefficients
```

```
  if (nrow(coefficient_summary) > 1) {  
    beta_coef <- coefficient_summary[2, 1]  
    std_error <- coefficient_summary[2, 2]  
    odds_ratio <- exp(beta_coef)  
    conf_lower <- exp(beta_coef - 1.96 * std_error)  
    conf_upper <- exp(beta_coef + 1.96 * std_error)  
    p_value <- coefficient_summary[2, 4]
```

```
    simple_logistic_models <- rbind(simple_logistic_models, data.frame(  
      Hormone = hormone,  
      Coefficient = round(beta_coef, 4),  
      Standard_Error = round(std_error, 4),
```

```

Odds_Ratio = round(odds_ratio, 3),
CI_Lower_95 = round(conf_lower, 3),
CI_Upper_95 = round(conf_upper, 3),
P_Value = round(p_value, 4),
AIC_Value = round(AIC(logistic_model), 2),
BIC_Value = round(BIC(logistic_model), 2),
stringsAsFactors = FALSE
))
}
}

# Sort by BIC (lower is better)

simple_logistic_models                                     <-
simple_logistic_models[order(simple_logistic_models$BIC_Value), ]

cat("\nSimple Logistic Regression Results (ordered by BIC):\n")
print(simple_logistic_models)

# Display significant findings

significant_predictors                                     <-
simple_logistic_models[simple_logistic_models$P_Value < 0.05, ]

if (nrow(significant_predictors) > 0) {
  cat("\nStatistically significant predictors (p < 0.05):\n")

```

```

    print(significant_predictors[, c("Hormone", "Odds_Ratio", "CI_Lower_95",
"CI_Upper_95", "P_Value")])

}

```

```

# -----

```

```

# MODEL 2: Multiple logistic regression (all hormones together)

```

```

# -----

```

```

cat("\n" , paste(rep("-", 40), collapse = ""), "\n")

```

```

cat("MODEL 2: Multiple Logistic Regression (All Hormones)\n")

```

```

cat(paste(rep("-", 40), collapse = ""), "\n")

```

```

full_formula <- as.formula(paste("outcome ~", paste(hormone_list, collapse = "
+ ")))

```

```

full_logistic_model <- glm(full_formula, data = logistic_dataset, family =
binomial)

```

```

# Calculate odds ratios for all predictors

```

```

full_coefficients <- summary(full_logistic_model)$coefficients

```

```

odds_ratios_all <- data.frame(

```

```

  Variable = rownames(full_coefficients),

```

```

  Coefficient = round(full_coefficients[, 1], 4),

```

```

  Standard_Error = round(full_coefficients[, 2], 4),

```

```

  Odds_Ratio = round(exp(full_coefficients[, 1]), 3),

```

```

CI_Lower = round(exp(full_coefficients[, 1] - 1.96 * full_coefficients[, 2]), 3),
CI_Upper = round(exp(full_coefficients[, 1] + 1.96 * full_coefficients[, 2]), 3),
P_Value = round(full_coefficients[, 4], 4),
stringsAsFactors = FALSE
)

cat("\nOdds Ratios for All Hormones Model:\n")
print(odds_ratios_all)

cat("\nModel Fit Statistics:\n")
cat(" AIC:", round(AIC(full_logistic_model), 2), "\n")
cat(" BIC:", round(BIC(full_logistic_model), 2), "\n")

# -----
# MODEL 3: Stepwise selection using BIC
# -----

cat("\n" , paste(rep("-", 40), collapse = ""), "\n")
cat("MODEL 3: Stepwise Logistic Regression (BIC Criterion)\n")
cat(paste(rep("-", 40), collapse = ""), "\n")

intercept_only_model <- glm(outcome ~ 1, data = logistic_dataset, family =
binomial)

```

```

stepwise_bic_model <- step(intercept_only_model,
                           scope = full_formula,
                           direction = "both",
                           trace = 0,
                           k = log(nrow(logistic_dataset))) # BIC penalty

```

```

selected_vars_bic <- names(coef(stepwise_bic_model))[-1]

```

```

if (length(selected_vars_bic) > 0) {
  cat("Variables selected by BIC stepwise procedure:\n")
  cat(paste(selected_vars_bic, collapse = ", "), "\n\n")
}

```

```

# Calculate odds ratios for selected model

```

```

stepwise_coefficients <- summary(stepwise_bic_model)$coefficients
odds_ratios_stepwise <- data.frame(
  Variable = rownames(stepwise_coefficients),
  Coefficient = round(stepwise_coefficients[, 1], 4),
  Standard_Error = round(stepwise_coefficients[, 2], 4),
  Odds_Ratio = round(exp(stepwise_coefficients[, 1]), 3),
  CI_Lower = round(exp(stepwise_coefficients[, 1] - 1.96 *
stepwise_coefficients[, 2]), 3),

```

```

    CI_Upper = round(exp(stepwise_coefficients[, 1] + 1.96 *
stepwise_coefficients[, 2]), 3),

    P_Value = round(stepwise_coefficients[, 4], 4),

    stringsAsFactors = FALSE

)

print(odds_ratios_stepwise)

cat("\nModel Fit Statistics:\n")

cat(" AIC:", round(AIC(stepwise_bic_model), 2), "\n")

cat(" BIC:", round(BIC(stepwise_bic_model), 2), "\n")

} else {

cat("Stepwise procedure selected no variables\n")

stepwise_bic_model <- intercept_only_model

}

# -----

# MODEL 4: Forward selection using AIC

# -----

cat("\n" , paste(rep("-", 40), collapse = ""), "\n")

cat("MODEL 4: Forward Selection Logistic Regression (AIC Criterion)\n")

cat(paste(rep("-", 40), collapse = ""), "\n")

```

```

forward_aic_model <- step(intercept_only_model,
                           scope = full_formula,
                           direction = "forward",
                           trace = 0,
                           k = 2) # AIC penalty

```

```

selected_vars_aic <- names(coef(forward_aic_model))[-1]

```

```

if (length(selected_vars_aic) > 0) {
  cat("Variables selected by AIC forward selection:\n")
  cat(paste(selected_vars_aic, collapse = ", "), "\n\n")
}

```

```

forward_coefficients <- summary(forward_aic_model)$coefficients
odds_ratios_forward <- data.frame(
  Variable = rownames(forward_coefficients),
  Odds_Ratio = round(exp(forward_coefficients[, 1]), 3),
  CI_Lower = round(exp(forward_coefficients[, 1] - 1.96 *
forward_coefficients[, 2]), 3),
  CI_Upper = round(exp(forward_coefficients[, 1] + 1.96 *
forward_coefficients[, 2]), 3),
  P_Value = round(forward_coefficients[, 4], 4),
  stringsAsFactors = FALSE
)

```

```

)

print(odds_ratios_forward)

cat("\nModel AIC:", round(AIC(forward_aic_model), 2), "\n")
cat("Model BIC:", round(BIC(forward_aic_model), 2), "\n")
}

# -----
# MODEL COMPARISON
# -----

cat("\n" , paste(rep("=", 40), collapse = ""), "\n")
cat("MODEL COMPARISON SUMMARY\n")
cat(paste(rep("=", 40), collapse = ""), "\n")

model_comparison_table <- data.frame(
  Model_Type = character(),
  AIC_Value = numeric(),
  BIC_Value = numeric(),
  Number_of_Predictors = integer(),
  stringsAsFactors = FALSE
)

```

```

# Add best simple model

if (nrow(simple_logistic_models) > 0) {

  best_simple_row                                     <-
simple_logistic_models[which.min(simple_logistic_models$BIC_Value), ]

  model_comparison_table <- rbind(model_comparison_table, data.frame(

    Model_Type = paste("Simple (", best_simple_row$Hormone, ")", sep = ""),

    AIC_Value = best_simple_row$AIC_Value,

    BIC_Value = best_simple_row$BIC_Value,

    Number_of_Predictors = 1,

    stringsAsFactors = FALSE

  ))

}

# Add full model

model_comparison_table <- rbind(model_comparison_table, data.frame(

  Model_Type = "All Hormones",

  AIC_Value = round(AIC(full_logistic_model), 2),

  BIC_Value = round(BIC(full_logistic_model), 2),

  Number_of_Predictors = length(hormone_list),

  stringsAsFactors = FALSE

))

```

```

# Add stepwise BIC model

if (exists("stepwise_bic_model") && length(coef(stepwise_bic_model)) > 1) {
  model_comparison_table <- rbind(model_comparison_table, data.frame(
    Model_Type = "Stepwise (BIC)",
    AIC_Value = round(AIC(stepwise_bic_model), 2),
    BIC_Value = round(BIC(stepwise_bic_model), 2),
    Number_of_Predictors = length(coef(stepwise_bic_model)) - 1,
    stringsAsFactors = FALSE
  ))
}

```

```

# Add forward AIC model

if (exists("forward_aic_model") && length(coef(forward_aic_model)) > 1) {
  model_comparison_table <- rbind(model_comparison_table, data.frame(
    Model_Type = "Forward (AIC)",
    AIC_Value = round(AIC(forward_aic_model), 2),
    BIC_Value = round(BIC(forward_aic_model), 2),
    Number_of_Predictors = length(coef(forward_aic_model)) - 1,
    stringsAsFactors = FALSE
  ))
}

```

```

# Add null model (intercept only)

```

```

model_comparison_table <- rbind(model_comparison_table, data.frame(
  Model_Type = "Null (Intercept Only)",
  AIC_Value = round(AIC(intercept_only_model), 2),
  BIC_Value = round(BIC(intercept_only_model), 2),
  Number_of_Predictors = 0,
  stringsAsFactors = FALSE
))

# Sort by BIC (lower is better)

model_comparison_table                                     <-
model_comparison_table[order(model_comparison_table$BIC_Value), ]

cat("\nModel Comparison (ordered by BIC - lower values indicate better fit):\n")
print(model_comparison_table)

# Identify the best model

best_model_row                                             <-
model_comparison_table[which.min(model_comparison_table$BIC_Value), ]

cat("\n BEST MODEL ACCORDING TO BIC:", best_model_row$Model_Type,
"\n")

cat(" BIC =", best_model_row$BIC_Value, "\n")

cat(" AIC =", best_model_row$AIC_Value, "\n")

cat(" Number of predictors =", best_model_row$Number_of_Predictors, "\n")

```

```

# -----

# MODEL PERFORMANCE EVALUATION

# -----

cat("\n" , paste(rep("=", 40), collapse = ""), "\n")
cat("MODEL PERFORMANCE METRICS\n")
cat(paste(rep("=", 40), collapse = ""), "\n")

# Function to calculate performance metrics
evaluate_model_performance <- function(model, dataset, model_name) {
  predicted_probabilities <- predict(model, newdata = dataset, type = "response")
  predicted_classes <- ifelse(predicted_probabilities > 0.5, 1, 0)
  actual_outcomes <- dataset$outcome

  # Confusion matrix
  confusion_matrix <- table(Predicted = predicted_classes, Actual =
actual_outcomes)

  # Calculate metrics
  accuracy_value <- sum(diag(confusion_matrix)) / sum(confusion_matrix)

  sensitivity_value <- NA
  specificity_value <- NA

```

```

if (nrow(confusion_matrix) == 2) {
  sensitivity_value <- confusion_matrix[2, 2] / sum(actual_outcomes == 1)
  specificity_value <- confusion_matrix[1, 1] / sum(actual_outcomes == 0)
}

# Brier score (lower is better)
brier_score <- mean((predicted_probabilities - actual_outcomes)^2)

# McFadden's pseudo R-squared
null_model <- glm(outcome ~ 1, data = dataset, family = binomial)
pseudo_r2 <- 1 - (logLik(model) / logLik(null_model))

cat("\n", model_name, ":\n", sep = "")
cat(" Prediction accuracy:", round(accuracy_value, 4), "\n")
cat(" Sensitivity (true positive rate):", round(sensitivity_value, 4), "\n")
cat(" Specificity (true negative rate):", round(specificity_value, 4), "\n")
cat(" Brier score (lower is better):", round(brier_score, 4), "\n")
cat(" Pseudo R-squared (McFadden):", round(as.numeric(pseudo_r2), 4), "\n")

return(c(Accuracy = accuracy_value,
        Sensitivity = sensitivity_value,
        Specificity = specificity_value,
        Brier = brier_score,

```

```

    Pseudo_R2 = as.numeric(pseudo_r2)))
  }

# Initialize performance results table
performance_summary <- data.frame(
  Model = character(),
  Accuracy = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric(),
  Brier_Score = numeric(),
  Pseudo_R_Squared = numeric(),
  stringsAsFactors = FALSE
)

# Evaluate best simple model
if (nrow(simple_logistic_models) > 0) {
  best_simple_hormone <-
simple_logistic_models$Hormone[which.min(simple_logistic_models$BIC_Value)
]
  best_simple_logistic <- glm(outcome ~ get(best_simple_hormone),
                             data = logistic_dataset, family = binomial)

  simple_performance <- evaluate_model_performance(best_simple_logistic,
logistic_dataset,

                                                     paste("Simple:", best_simple_hormone))

```

```

performance_summary <- rbind(performance_summary, data.frame(
  Model = paste("Simple:", best_simple_hormone),
  Accuracy = simple_performance["Accuracy"],
  Sensitivity = simple_performance["Sensitivity"],
  Specificity = simple_performance["Specificity"],
  Brier_Score = simple_performance["Brier"],
  Pseudo_R_Squared = simple_performance["Pseudo_R2"],
  stringsAsFactors = FALSE
))
}

```

Evaluate full model

```

full_performance <- evaluate_model_performance(full_logistic_model,
logistic_dataset, "All Hormones")

performance_summary <- rbind(performance_summary, data.frame(
  Model = "All Hormones",
  Accuracy = full_performance["Accuracy"],
  Sensitivity = full_performance["Sensitivity"],
  Specificity = full_performance["Specificity"],
  Brier_Score = full_performance["Brier"],
  Pseudo_R_Squared = full_performance["Pseudo_R2"],
  stringsAsFactors = FALSE
))

```

```

# Evaluate stepwise model

if (exists("stepwise_bic_model") && length(coef(stepwise_bic_model)) > 1) {

  stepwise_performance <- evaluate_model_performance(stepwise_bic_model,
logistic_dataset, "Stepwise (BIC)")

  performance_summary <- rbind(performance_summary, data.frame(

    Model = "Stepwise (BIC)",

    Accuracy = stepwise_performance["Accuracy"],

    Sensitivity = stepwise_performance["Sensitivity"],

    Specificity = stepwise_performance["Specificity"],

    Brier_Score = stepwise_performance["Brier"],

    Pseudo_R_Squared = stepwise_performance["Pseudo_R2"],

    stringsAsFactors = FALSE

  ))

}

```

```

# Evaluate forward model

if (exists("forward_aic_model") && length(coef(forward_aic_model)) > 1) {

  forward_performance <- evaluate_model_performance(forward_aic_model,
logistic_dataset, "Forward (AIC)")

  performance_summary <- rbind(performance_summary, data.frame(

    Model = "Forward (AIC)",

    Accuracy = forward_performance["Accuracy"],


```

```

Sensitivity = forward_performance["Sensitivity"],
Specificity = forward_performance["Specificity"],
Brier_Score = forward_performance["Brier"],
Pseudo_R_Squared = forward_performance["Pseudo_R2"],
stringsAsFactors = FALSE

))
}

# Sort by accuracy

performance_summary <- performance_summary[order(-
performance_summary$Accuracy), ]

cat("\n\nPerformance Summary (ordered by prediction accuracy):\n")
print(performance_summary)

# -----

# SAVE LOGISTIC REGRESSION RESULTS

# -----

write.csv(simple_logistic_models, "03_logistic_simple_models.csv", row.names
= FALSE)

write.csv(odds_ratios_all, "03_logistic_odds_ratios_all_hormones.csv",
row.names = FALSE)

```

```
write.csv(model_comparison_table, "03_logistic_model_comparison.csv",
row.names = FALSE)
```

```
write.csv(performance_summary, "03_logistic_model_performance.csv",
row.names = FALSE)
```

```
if (exists("odds_ratios_stepwise")) {
  write.csv(odds_ratios_stepwise, "03_logistic_odds_ratios_stepwise.csv",
row.names = FALSE)
}
```

```
cat("\n Logistic regression results saved to CSV files\n")
}
}
```

```
# -----
```

```
# SECTION 6: FINAL SUMMARY REPORT
```

```
# -----
```

```
# Create comprehensive text summary
```

```
sink("00_COMPLETE_ANALYSIS_REPORT.txt")
```

```
cat(paste(rep("=", 60), collapse = ""), "\n")
```

```
cat("COMPLETE STATISTICAL ANALYSIS REPORT\n")
```

```
cat(paste(rep("=", 60), collapse = ""), "\n\n")
```

```
cat("Analysis Date:", date(), "\n\n")
```

```
cat("DATA SUMMARY\n")
```

```
cat(paste(rep("-", 40), collapse = ""), "\n")
```

```
cat("Total observations:", nrow(study_data), "\n")
```

```
cat("Total variables:", ncol(study_data), "\n\n")
```

```
cat("VARIABLES INCLUDED IN ANALYSIS\n")
```

```
cat(paste(rep("-", 40), collapse = ""), "\n")
```

```
cat("Hormone variables:", length(hormone_list), "\n")
```

```
cat("Lipid variables:", length(lipid_list), "\n")
```

```
cat("Lipid peroxidation markers:", length(peroxidation_list), "\n")
```

```
cat("Antioxidant markers:", length(antioxidant_list), "\n")
```

```
cat("Clinical factors:", length(clinical_list), "\n\n")
```

```
# Correlation section
```

```
cat(paste(rep("=", 60), collapse = ""), "\n")
```

```
cat("SECTION 1: CORRELATION ANALYSIS RESULTS\n")
```

```
cat(paste(rep("=", 60), collapse = ""), "\n\n")
```

```
cat("Total correlation pairs calculated:", nrow(correlation_table), "\n")
```

```
cat("Statistically significant pairs ( $p < 0.05$ ):", nrow(significant_pairs), "\n")
```

```
cat("\nTop 10 strongest correlations:\n")
```

```
print(head(correlation_table[, c("Variable_A", "Variable_B",
"Correlation_Coefficient", "Significance_Flag")], 10))
```

```
# Regression section
```

```
cat("\n\n", paste(rep("=", 60), collapse = ""), "\n")
```

```
cat("SECTION 2: REGRESSION ANALYSIS RESULTS\n")
```

```
cat(paste(rep("=", 60), collapse = ""), "\n\n")
```

```
if (nrow(regression_summary_table) > 0) {
```

```
  cat("Total regression models fitted:", nrow(regression_summary_table), "\n")
```

```
  cat("\nBest model for each variable category:\n")
```

```
  for (var_category in unique(regression_summary_table$Variable_Category)) {
```

```
    category_best <-
```

```
    regression_summary_table[regression_summary_table$Variable_Category ==
    var_category, ]
```

```
    category_best <- category_best[order(category_best$BIC_Value), ]
```

```
    if (nrow(category_best) > 0) {
```

```
      cat("\n", var_category, ":\n", sep = "")
```

```
      cat(" Best model for:", category_best$Response_Variable[1], "\n")
```

```
      cat(" R-squared:", category_best$R_Squared[1], "\n")
```

```
      cat(" BIC:", category_best$BIC_Value[1], "\n")
```

```
    }
```

```
  }
```

```

cat("\n OVERALL BEST REGRESSION MODEL:\n")

overall_best <-
regression_summary_table[which.min(regression_summary_table$BIC_Value), ]

cat(" Response variable:", overall_best$Response_Variable, "\n")

cat(" R-squared:", overall_best$R_Squared, "\n")

cat(" Adjusted R-squared:", overall_best$Adjusted_R_Squared, "\n")

cat(" BIC:", overall_best$BIC_Value, "\n")

cat(" Selected predictors:", overall_best$Selected_Predictors, "\n")
}

# Logistic regression section

cat("\n\n", paste(rep("=", 60), collapse = ""), "\n")

cat("SECTION 3: LOGISTIC REGRESSION RESULTS\n")

cat(paste(rep("=", 60), collapse = ""), "\n\n")

if (exists("model_comparison_table") && nrow(model_comparison_table) > 0) {
  cat("Total logistic models compared:", nrow(model_comparison_table), "\n")
  cat("\nBest model:", best_model_row$Model_Type, "\n")
  cat(" BIC:", best_model_row$BIC_Value, "\n")
  cat(" AIC:", best_model_row$AIC_Value, "\n")
  cat(" Predictors:", best_model_row$Number_of_Predictors, "\n")
}

```

```
cat("\nModel performance summary:\n")  
print(performance_summary)  
}
```

```
cat("\n\n", paste(rep("=", 60), collapse = ""), "\n")  
cat("END OF REPORT\n")  
cat(paste(rep("=", 60), collapse = ""), "\n")
```